Another Mother — Stripe Connect Implementation Guide

Version 1.0 | MVP Phase 1

Platform: Australian Marketplace | Currency: AUD

---

1. STRIPE CONNECT ARCHITECTURE OVERVIEW

1.1 Why Stripe Connect (Not Standard Stripe)

Requirement	Standard Stripe	Stripe Connect
Split payments (85/15)	❌ Manual reconciliation	✅ Automatic split on charge
Hold funds in escrow	❌ Immediate payout to platform	✅ Hold on platform, transfer later
Provider onboarding (KYC)	❌ Not supported	✅ Built-in identity verification
Provider gets own dashboard	❌ Not supported	✅ Optional Express/Custom account
Marketplace fee collection	❌ Manual invoicing	✅ Automatic platform fee retention
Australian compliance	Partial	Full (AU GST, tax reporting)

Another Mother uses Stripe Connect with Express accounts — providers get a simple onboarding flow, you get full control over timing of payouts.

1.2 Account Structure

```
Another Mother Platform Account (acct_platform_xxxxx)
├── Platform Balance (holds all family payments)
│   ├── 15% Platform Fee → Another Mother revenue
│   └── 85% Provider Payout → Held until job completion
│
└── Connected Accounts (one per provider)
    ├── Sarah M. (acct_1P2xYzAbCdEfGhIj)
    │   └── Receives 85% on job completion
    ├── Jane D. (acct_1P3xYzAbCdEfGhIj)
    │   └── Receives 85% on job completion
    └── ...
```

1.3 Connect Account Type: Express

Type	Best For	Another Mother?
Express	Marketplaces, quick onboarding, platform-controlled payouts	✅ **YES** —Providers onboard in <5 min
Standard	Full control to connected account, separate Stripe login	❌ Too complex for mums
Custom	Full platform control, no Stripe UI	❌ High compliance burden

Express chosen because:

- Providers don't need a separate Stripe login
- Onboarding is embedded in your app
- You control payout timing (daily, weekly, instant)
- Stripe handles KYC/AML compliance
- Lower fraud risk (Stripe verifies identity)

---

2. CONNECT ONBOARDING FLOW

2.1 Provider Onboarding Steps

```
Provider completes profile → Clicks "Get Paid" → Stripe Connect onboarding
↓
Stripe.js opens Express onboarding
↓
Provider provides:
  • Identity verification (driver licence/passport)
  • Bank account details (BSB + Account Number)
  • Date of birth
  • Business type (Individual/Sole Trader)
  • Tax information (TFN for Australian tax reporting)
↓
Stripe verifies identity (automated, <2 min usually)
↓
Account status: "pending" → "active" (or "restricted" if more info needed)
↓
Provider can now receive job offers
```

2.2 API: Create Connect Account

Endpoint: `POST /v1/accounts` (Stripe API, server-side)

Request:

```
{
  "type": "express",
  "country": "AU",
  "email": "sarah.m@email.com",
  "capabilities": {
    "transfers": {"requested": true},
    "card_payments": {"requested": false}
  },
  "business_type": "individual",
  "business_profile": {
    "name": "Sarah M. — Another Mother Provider",
    "url": "https://anothermother.com.au",
    "product_description": "Providing childcare and household support services to local families"
  },
  "settings": {
    "payouts": {
```

```
      "schedule": {
        "interval": "weekly",
        "weekly_anchor": "thursday"
      }
    }
  },
  "metadata": {
    "provider_id": "770e8400-e29b-41d4-a716-446655440003",
    "platform": "another_mother"
  }
}
```

Response:

```
{
  "id": "acct_1P2xYzAbCdEfGhIj",
  "type": "express",
  "charges_enabled": false,
  "payouts_enabled": false,
  "requirements": {
    "currently_due": ["external_account", "individual.dob", "individual.first_name",
    ↪  "individual.last_name"],
    "eventually_due": ["individual.verification.document"],
    "past_due": [],
    "pending_verification": []
  },
  "settings": {
    "payouts": {
      "schedule": {
        "interval": "weekly",
        "weekly_anchor": "thursday"
      }
    }
  }
}
```

Store in your database:

```
UPDATE provider_profiles
SET stripe_connect_account_id = 'acct_1P2xYzAbCdEfGhIj',
    stripe_connect_status = 'pending_onboarding'
WHERE id = '770e8400-e29b-41d4-a716-446655440003';
```

2.3 API: Generate Onboarding Link

Endpoint: `POST /v1/account_links` (Stripe API, server-side)

Request:

```
{
  "account": "acct_1P2xYzAbCdEfGhIj",
  "refresh_url": "https://anothermother.com.au/provider/onboarding?status=refresh",
```

```
  "return_url": "https://anothermother.com.au/provider/onboarding?status=success",
  "type": "account_onboarding",
  "collect": "eventually_due"
}
```

Response:

```
{
  "object": "account_link",
  "url": "https://connect.stripe.com/setup/s/acct_1P2xYzAbCdEfGhIj/...",
  "expires_at": 1715629200
}
```

Frontend flow:

```
// Provider clicks "Complete Setup to Get Paid"
window.location.href = accountLink.url;

// After Stripe onboarding, user redirected to:
// https://anothermother.com.au/provider/onboarding?status=success
// → Your frontend calls GET /providers/me to check stripe_connect_status
```

2.4 Onboarding Status Tracking

Status	Meaning	Action
`pending_onboarding`	Account created, onboarding not started	Show “Complete setup” button
`pending_verification`	Info submitted, Stripe reviewing	Show “Under review —usually 24-48 hours”
`restricted`	More info needed	Show specific requirements from Stripe
`active`	Ready to receive payouts	Show “You’ re all set!” + earnings dashboard
`suspended`	Compliance issue	Admin intervention required

Webhook event that triggers status update: `account.updated`

---

3. PAYMENT FLOW: ESCROW PATTERN

3.1 The Full Job Payment Lifecycle

```
FAMILY BOOKS JOB
↓
[1] Create PaymentIntent (family card charged, funds held on platform)
    amount: $32.20 ($28.00 service + $4.20 platform fee)
    capture_method: manual (hold, don't capture yet)
    transfer_group: "job_aa0e8400-e29b-41d4-a716-446655440006"
    on_behalf_of: null (platform is merchant of record)
```

```
↓
[2] Family sees: "Payment held securely. You won't be charged until job is complete."
↓
PROVIDER ACCEPTS JOB
↓
[3] Job status: confirmed
    PaymentIntent status: requires_capture (still holding)
↓
JOB COMPLETED
↓
[4] Provider marks complete → Family verifies
↓
[5] CAPTURE PAYMENT + CREATE TRANSFER
    POST /v1/payment_intents/{pi_id}/capture
    ↓
    POST /v1/transfers
    {
      amount: 2800,  // $28.00 in cents
      currency: "aud",
      destination: "acct_1P2xYzAbCdEfGhIj",
      transfer_group: "job_aa0e8400-e29b-41d4-a716-446655440006",
      metadata: { job_id: "...", provider_id: "..." }
    }
↓
[6] Platform retains $4.20 (15%) as revenue
    Provider receives $28.00 in their Stripe balance
↓
[7] Provider payout scheduled (weekly Thursday, or instant if requested)
```

3.2 API: Create PaymentIntent (Step 1)

Endpoint: `POST /v1/payment_intents` (Stripe API, server-side)

Request:

```
{
  "amount": 3220,
  "currency": "aud",
  "capture_method": "manual",
  "confirmation_method": "manual",
  "confirm": false,
  "transfer_group": "job_aa0e8400-e29b-41d4-a716-446655440006",
  "metadata": {
    "job_id": "aa0e8400-e29b-41d4-a716-446655440006",
    "family_id": "550e8400-e29b-41d4-a716-446655440000",
    "service_category": "school-pickups",
    "platform_fee_cents": 420,
    "service_amount_cents": 2800
  },
  "receipt_email": "sarah.m@email.com",
  "statement_descriptor": "Another Mother",
  "statement_descriptor_suffix": "School Pickup",
```

```
  "automatic_payment_methods": {
    "enabled": true,
    "allow_redirects": "never"
  }
}
```

Response:

```
{
  "id": "pi_3P2xYzAbCdEfGhIj_secret_xyz",
  "object": "payment_intent",
  "amount": 3220,
  "amount_capturable": 0,
  "amount_received": 0,
  "capture_method": "manual",
  "client_secret": "pi_3P2xYzAbCdEfGhIj_secret_xyz",
  "confirmation_method": "manual",
  "currency": "aud",
  "status": "requires_confirmation",
  "transfer_group": "job_aa0e8400-e29b-41d4-a716-446655440006",
  "metadata": {...}
}
```

Store in your database:

```
UPDATE jobs
SET stripe_payment_intent_id = 'pi_3P2xYzAbCdEfGhIj_secret_xyz',
    payment_status = 'hold_pending'
WHERE id = 'aa0e8400-e29b-41d4-a716-446655440006';
```

3.3 Frontend: Confirm Payment (Stripe.js)

```
// Family enters card details via Stripe Elements
const {paymentIntent, error} = await stripe.confirmCardPayment(
  clientSecret,  // From backend: pi_3P2xYzAbCdEfGhIj_secret_xyz
  {
    payment_method: {
      card: cardElement,
      billing_details: {
        name: "Sarah Mitchell",
        email: "sarah.m@email.com",
        address: {
          line1: "123 Given Terrace",
          city: "Paddington",
          state: "QLD",
          postal_code: "4064",
          country: "AU"
        }
      }
    },
    setup_future_usage: 'off_session'  // For recurring bookings
```

```
  }
);

// On success: paymentIntent.status === "requires_capture"
// → Frontend calls POST /jobs/{id}/confirm-payment to your backend
```

3.4 API: Capture Payment + Transfer to Provider (Step 5)

This is the CRITICAL escrow release moment.

Step 5a: Capture the hold

```
POST /v1/payment_intents/pi_3P2xYzAbCdEfGhIj/capture
{
  "amount_to_capture": 3220,  // Full amount
  "statement_descriptor": "Another Mother - Job Complete"
}
```

Response:

```
{
  "id": "pi_3P2xYzAbCdEfGhIj",
  "status": "succeeded",
  "charges": {
    "data": [{
      "id": "ch_3P2xYzAbCdEfGhIj",
      "balance_transaction": "txn_3P2xYzAbCdEfGhIj",
      "amount": 3220,
      "currency": "aud"
    }]
  }
}
```

Step 5b: Transfer 85% to provider

```
POST /v1/transfers
{
  "amount": 2800,
  "currency": "aud",
  "destination": "acct_1P2xYzAbCdEfGhIj",
  "transfer_group": "job_aa0e8400-e29b-41d4-a716-446655440006",
  "source_transaction": "ch_3P2xYzAbCdEfGhIj",
  "metadata": {
    "job_id": "aa0e8400-e29b-41d4-a716-446655440006",
    "provider_id": "770e8400-e29b-41d4-a716-446655440003",
    "service_amount_cents": 2800,
    "platform_fee_cents": 420
  }
}
```

Response:

```
{
```

```
  "id": "tr_3P2xYzAbCdEfGhIj",
  "object": "transfer",
  "amount": 2800,
  "currency": "aud",
  "destination": "acct_1P2xYzAbCdEfGhIj",
  "transfer_group": "job_aa0e8400-e29b-41d4-a716-446655440006",
  "source_transaction": "ch_3P2xYzAbCdEfGhIj",
  "status": "paid"
}
```

Store in your database:

```
UPDATE jobs
SET status = 'verified_complete',
    stripe_transfer_id = 'tr_3P2xYzAbCdEfGhIj',
    payment_status = 'completed',
    completed_at = NOW(),
    verified_complete_at = NOW()
WHERE id = 'aa0e8400-e29b-41d4-a716-446655440006';

INSERT INTO transactions (job_id, transaction_type, from_party, to_party, amount,
  currency, stripe_payment_intent_id, stripe_transfer_id, status, description)
VALUES
  ('aa0e8400-e29b-41d4-a716-446655440006', 'payment_hold', null, null, 32.20, 'AUD',
   'pi_3P2xYzAbCdEfGhIj', null, 'completed', 'Family payment captured'),
  ('aa0e8400-e29b-41d4-a716-446655440006', 'provider_payout', null,
   ↪  '770e8400-e29b-41d4-a716-446655440003', 28.00, 'AUD',
   'pi_3P2xYzAbCdEfGhIj', 'tr_3P2xYzAbCdEfGhIj', 'completed', 'Provider payout'),
  ('aa0e8400-e29b-41d4-a716-446655440006', 'platform_fee', null, null, 4.20, 'AUD',
   'pi_3P2xYzAbCdEfGhIj', null, 'completed', 'Platform fee (15%)');
```

---

4. WEBHOOK HANDLING

4.1 Webhook Endpoint Setup

Your endpoint: `POST https://api.anothermother.com.au/v1/webhooks/stripe`

Stripe Dashboard → Developers → Webhooks → Add endpoint:

- URL: `https://api.anothermother.com.au/v1/webhooks/stripe`
- Events to listen for: (see below)
- Secret: `whsec_xxxxxxxxxxxxxxxx` (for signature verification)

4.2 Required Webhook Events

Event	When It Fires	Your Action
`payment_intent.amount_capturable_updated`	Hold placed, funds authorised	Update job status to `confirmed`, notify provider

Table 4 – continued

Event	When It Fires	Your Action
`payment_intent.succeeded`	Payment captured successfully	Log transaction, trigger provider payout
`payment_intent.payment_failed`	Card declined or hold expired	Cancel job, notify family, offer retry
`payment_intent.canceled`	Family or system cancelled hold	Release hold, update job status
`transfer.created`	Transfer to provider initiated	Log in transactions table
`transfer.paid`	Transfer reached provider's Stripe balance	Update provider earnings, notify provider
`payout.created`	Provider payout (to bank) initiated	Log in payout_history
`payout.paid`	Provider payout reached bank account	Update payout status, notify provider
`payout.failed`	Bank rejected payout	Alert admin, notify provider, investigate
`account.updated`	Provider Connect account status changes	Update provider verification status
`account.external_account.updated`	Provider changes bank details	Log for audit
`charge.dispute.created`	Family disputes charge	Immediately freeze job, alert admin
`charge.refunded`	Refund processed	Update job status, log transaction
`invoice.payment_failed`	Subscription payment failed (Post-MVP)	Grace period, notify user

4.3 Webhook Handler Implementation (Node.js Example)

```
const stripe = require('stripe')(process.env.STRIPE_SECRET_KEY);
const express = require('express');
const router = express.Router();

// Webhook endpoint — MUST use raw body, not JSON parser
router.post('/webhooks/stripe',
  express.raw({ type: 'application/json' }),
  async (req, res) => {
    const sig = req.headers['stripe-signature'];
    const endpointSecret = process.env.STRIPE_WEBHOOK_SECRET;

    let event;
    try {
      event = stripe.webhooks.constructEvent(req.body, sig, endpointSecret);
    } catch (err) {
      console.error(`Webhook signature verification failed: ${err.message}`);
      return res.status(400).send(`Webhook Error: ${err.message}`);
    }

    // Acknowledge receipt immediately (Stripe retries if 5xx)
    res.status(200).json({ received: true });

    // Process event asynchronously
    try {
      await handleStripeEvent(event);
    } catch (err) {
      console.error(`Error processing event ${event.id}:`, err);
```

```
      // Log to error tracking (Sentry, etc.)
      // Stripe will retry automatically if we return 5xx, but we already sent 200
      // So we must handle retries manually via dead letter queue
    }
  }
);

async function handleStripeEvent(event) {
  const { type, data } = event;

  switch (type) {
    case 'payment_intent.amount_capturable_updated':
      await handlePaymentHoldPlaced(data.object);
      break;

    case 'payment_intent.succeeded':
      await handlePaymentSucceeded(data.object);
      break;

    case 'payment_intent.payment_failed':
      await handlePaymentFailed(data.object);
      break;

    case 'transfer.paid':
      await handleTransferPaid(data.object);
      break;

    case 'payout.paid':
      await handlePayoutPaid(data.object);
      break;

    case 'payout.failed':
      await handlePayoutFailed(data.object);
      break;

    case 'account.updated':
      await handleAccountUpdated(data.object);
      break;

    case 'charge.dispute.created':
      await handleDisputeCreated(data.object);
      break;

    case 'charge.refunded':
      await handleChargeRefunded(data.object);
      break;

    default:
      console.log(`Unhandled event type: ${type}`);
  }
}
```

```
// [1] Payment hold placed — job can now be confirmed
async function handlePaymentHoldPlaced(paymentIntent) {
  const jobId = paymentIntent.metadata.job_id;

  await db.query(`
    UPDATE jobs
    SET payment_status = 'held',
        status = CASE WHEN status = 'offered' THEN 'confirmed' ELSE status END
    WHERE id = $1
  `, [jobId]);

  // Notify provider: "Payment secured. Job is confirmed!"
  await notifyProvider(jobId, 'payment_secured');
}

// [2] Payment captured — release to provider
async function handlePaymentSucceeded(paymentIntent) {
  const jobId = paymentIntent.metadata.job_id;
  const providerId = paymentIntent.metadata.provider_id;

  // Transfer already created in our API call, but webhook confirms it
  await db.query(`
    UPDATE transactions
    SET status = 'completed',
        processed_at = NOW()
    WHERE job_id = $1 AND transaction_type = 'payment_hold'
  `, [jobId]);

  // Update provider earnings
  await db.query(`
    UPDATE provider_earnings
    SET pending_balance = pending_balance - 28.00,
        available_balance = available_balance + 28.00,
        lifetime_earnings = lifetime_earnings + 28.00,
        updated_at = NOW()
    WHERE provider_id = $1
  `, [providerId]);

  // SendGrid: "You've been paid $28.00 for School Pickup on May 20"
  await sendProviderPayoutNotification(providerId, jobId);
}

// [3] Payment failed — cancel job, notify family
async function handlePaymentFailed(paymentIntent) {
  const jobId = paymentIntent.metadata.job_id;
  const failureMessage = paymentIntent.last_payment_error?.message || 'Payment failed';

  await db.query(`
    UPDATE jobs
    SET status = 'cancelled',
```

```
        payment_status = 'failed',
        cancellation_reason = $2,
        cancelled_at = NOW()
    WHERE id = $1
  `, [jobId, `Payment failed: ${failureMessage}`]);

  // Notify family: "Your payment couldn't be processed. Please update your card and try again."
  await notifyFamily(jobId, 'payment_failed', { reason: failureMessage });

  // If provider already assigned, release them
  await releaseProviderFromJob(jobId);
}

// [4] Transfer reached provider's Stripe balance
async function handleTransferPaid(transfer) {
  const jobId = transfer.metadata.job_id;

  await db.query(`
    UPDATE transactions
    SET status = 'completed',
        processed_at = NOW()
    WHERE stripe_transfer_id = $1
  `, [transfer.id]);
}

// [5] Payout reached provider's bank account
async function handlePayoutPaid(payout) {
  const providerAccountId = payout.destination;
  const provider = await db.query(`
    SELECT id FROM provider_profiles WHERE stripe_connect_account_id = $1
  `, [providerAccountId]);

  await db.query(`
    UPDATE payout_history
    SET status = 'paid',
        completed_at = NOW()
    WHERE stripe_payout_id = $1
  `, [payout.id]);

  // Notify provider: "$142.00 has landed in your bank account"
  await notifyProvider(provider.rows[0].id, 'payout_paid', {
    amount: payout.amount / 100,
    arrival_date: payout.arrival_date
  });
}

// [6] Payout failed — critical alert
async function handlePayoutFailed(payout) {
  const providerAccountId = payout.destination;
  const failureCode = payout.failure_code;
  const failureMessage = payout.failure_message;
```

```
  // Log incident
  await db.query(`
    INSERT INTO incidents (reporter_id, incident_type, severity, description, status)
    VALUES (
      (SELECT user_id FROM provider_profiles WHERE stripe_connect_account_id = $1),
      'payment_dispute',
      'high',
      $2,
      'open'
    )
  `, [providerAccountId, `Payout failed: ${failureCode} - ${failureMessage}`]);

  // Alert admin immediately
  await alertAdmin('payout_failed', {
    providerAccountId,
    payoutId: payout.id,
    failureCode,
    failureMessage
  });

  // Notify provider: "There was an issue with your payout. Our team is looking into it."
  await notifyProviderByAccountId(providerAccountId, 'payout_failed');
}

// [7] Provider account status changed
async function handleAccountUpdated(account) {
  const provider = await db.query(`
    SELECT id FROM provider_profiles WHERE stripe_connect_account_id = $1
  `, [account.id]);

  if (!provider.rows.length) return;

  const providerId = provider.rows[0].id;
  const chargesEnabled = account.charges_enabled;
  const payoutsEnabled = account.payouts_enabled;
  const requirements = account.requirements;

  let newStatus = 'pending_onboarding';
  if (chargesEnabled && payoutsEnabled) {
    newStatus = 'active';
  } else if (requirements.currently_due.length > 0) {
    newStatus = 'restricted';
  } else if (requirements.pending_verification.length > 0) {
    newStatus = 'pending_verification';
  }

  await db.query(`
    UPDATE provider_profiles
    SET stripe_connect_status = $1,
        verification_status = CASE
```

```
          WHEN $1 = 'active' THEN 'approved'
          WHEN $1 = 'restricted' THEN 'in_review'
          ELSE verification_status
        END,
        updated_at = NOW()
    WHERE id = $2
  `, [newStatus, providerId]);

  // Notify provider of status change
  await notifyProvider(providerId, 'account_status_changed', { status: newStatus });
}

// [8] Family disputes charge — freeze everything
async function handleDisputeCreated(charge) {
  const paymentIntentId = charge.payment_intent;
  const job = await db.query(`
    SELECT id, provider_id, family_id FROM jobs WHERE stripe_payment_intent_id = $1
  `, [paymentIntentId]);

  if (!job.rows.length) return;

  const { id: jobId, provider_id, family_id } = job.rows[0];

  // Freeze job
  await db.query(`
    UPDATE jobs SET status = 'disputed' WHERE id = $1
  `, [jobId]);

  // Create high-priority incident
  await db.query(`
    INSERT INTO incidents (reporter_id, incident_type, severity, job_id, description, status)
    VALUES ($1, 'payment_dispute', 'critical', $2, $3, 'open')
  `, [family_id, jobId, `Charge disputed for job ${jobId}. Amount: $${charge.amount / 100}`]);

  // Alert admin URGENT
  await alertAdmin('charge_dispute', {
    jobId,
    familyId: family_id,
    providerId: provider_id,
    amount: charge.amount / 100,
    disputeId: charge.dispute
  });

  // Notify both parties (carefully worded)
  await notifyFamily(family_id, 'dispute_acknowledged');
  await notifyProvider(provider_id, 'dispute_frozen', { jobId });
}

// [9] Refund processed
async function handleChargeRefunded(charge) {
  const paymentIntentId = charge.payment_intent;
```

```
  const refundAmount = charge.refunds.data[0].amount / 100;

  await db.query(`
    INSERT INTO transactions (job_id, transaction_type, amount, currency,
      stripe_payment_intent_id, status, description)
    SELECT id, 'refund', $2, 'AUD', $1, 'completed', 'Refund processed'
    FROM jobs WHERE stripe_payment_intent_id = $1
  `, [paymentIntentId, refundAmount]);
}
```

4.4 Webhook Security Checklist

Check	Implementation
✅ Signature verification	`stripe.webhooks.constructEvent()` with `whsec_` secret
✅ Raw body parsing	`express.raw({ type: 'application/json' })` —NOT `express.json()`
✅ Immediate 200 response	Send 200 before processing to prevent Stripe retries
✅ Idempotency	Check `event.id` against processed_events table before handling
✅ Async processing	Process event in background job, not in request handler
✅ Error handling	Log failures, dead letter queue for manual retry
✅ IP allowlisting	Stripe IPs: https://stripe.com/docs/ips

Idempotency table:

```
CREATE TABLE processed_webhook_events (
  event_id VARCHAR(100) PRIMARY KEY,
  event_type VARCHAR(100) NOT NULL,
  processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  status VARCHAR(20) DEFAULT 'success' CHECK (status IN ('success', 'failed', 'retrying'))
);
```

---

5. PAYOUT SCHEDULING

5.1 Default Payout: Weekly Auto (Thursday)

Stripe Connect settings per provider:

```
{
  "settings": {
    "payouts": {
      "schedule": {
        "interval": "weekly",
        "weekly_anchor": "thursday",
        "delay_days": 2  // 2-day hold for risk mitigation
      }
    }
  }
}
```

What this means:

- Job completed Monday → Provider balance updated Monday
- Payout initiated Thursday (weekly anchor)
- Funds in bank account Friday/Saturday (2-day delay + weekend)

5.2 Instant Payout (Provider-Requested)

When provider clicks "Cash Out Now":

```
// Backend creates instant payout
const payout = await stripe.payouts.create({
  amount: 14200,  // $142.00 in cents
  currency: 'aud',
  method: 'instant',  // vs 'standard'
  description: 'Instant payout - Another Mother',
  metadata: {
    provider_id: '770e8400-e29b-41d4-a716-446655440003',
    payout_type: 'instant',
    requested_by: 'provider'
  }
}, {
  stripeAccount: 'acct_1P2xYzAbCdEfGhIj'  // Provider's Connect account
});
```

Fees:

- Standard payout (weekly): **Free**
- Instant payout: **1.5% of payout amount** (Stripe fee)
- Example: $142.00 instant payout → $2.13 fee → $139.87 received

Store in database:

```
INSERT INTO payout_history (provider_id, amount, payout_method, stripe_payout_id, status, initiated_at)
VALUES ('770e8400-e29b-41d4-a716-446655440003', 142.00, 'instant', 'po_3P2xYzAbCdEfGhIj', 'pending',
↪  NOW());

UPDATE provider_earnings
SET available_balance = available_balance - 142.00,
    updated_at = NOW()
WHERE provider_id = '770e8400-e29b-41d4-a716-446655440003';
```

5.3 Payout Failure Handling

Failure Code	Meaning	Action
`account_closed`	Provider closed bank account	Notify provider, request new bank details
`no_account`	Bank account no longer exists	Same as above
`invalid_account_number`	BSB/account number invalid	Request correction

Table 6 – continued

Failure Code	Meaning	Action
`debit_not_authorized`	Bank rejects Stripe debits	Contact provider, alternative payout method
`insufficient_funds`	Provider' s Stripe balance negative	Investigate, likely refund/ chargeback

---

6. REFUND & CANCELLATION FLOW

6.1 Cancellation Timeline & Refund Rules

Cancellation Window	Family Refund	Provider Compensation	Platform Fee
Within 10 min of booking	100%	$0	$0
24h+ before job	100%	$0	$0
2-24h before job	50%	25% of service amount	25% of platform fee
<2h before job	0%	50% of service amount	0%
Provider cancels (any time)	100%	$0 + strike recorded	$0
No-show (provider)	100%	$0 + strike + suspension review	$0

6.2 API: Process Refund

Example: Family cancels 3 hours before job

```
// Calculate refund amounts
const serviceAmount = 2800;  // $28.00 in cents
const platformFee = 420;     // $4.20 in cents
const totalAmount = 3220;    // $32.20 in cents

// 50% refund to family
const familyRefund = Math.round(totalAmount * 0.5);  // 1610 cents = $16.10

// 25% to provider as compensation
const providerCompensation = Math.round(serviceAmount * 0.25);  // 700 cents = $7.00

// 25% platform fee retained
const platformFeeRetained = Math.round(platformFee * 0.25);  // 105 cents = $1.05

// Execute refund
const refund = await stripe.refunds.create({
  payment_intent: 'pi_3P2xYzAbCdEfGhIj',
  amount: familyRefund,
  reason: 'requested_by_customer',
  metadata: {
    job_id: 'aa0e8400-e29b-41d4-a716-446655440006',
    cancellation_reason: 'family_emergency',
    refund_type: 'partial_cancellation',
    provider_compensation_cents: providerCompensation
```

```
  }
});

// Transfer compensation to provider (from platform balance, not refund)
const compensationTransfer = await stripe.transfers.create({
  amount: providerCompensation,
  currency: 'aud',
  destination: 'acct_1P2xYzAbCdEfGhIj',
  transfer_group: 'job_aa0e8400-e29b-41d4-a716-446655440006',
  metadata: {
    type: 'cancellation_compensation',
    job_id: 'aa0e8400-e29b-41d4-a716-446655440006',
    reason: 'family_cancelled_within_24h'
  }
});
```

Database updates:

```
UPDATE jobs
SET status = 'cancelled',
    cancelled_by = '550e8400-e29b-41d4-a716-446655440000',
    cancelled_at = NOW(),
    cancellation_reason = 'family_emergency',
    cancellation_fee_applied = 16.10,
    payment_status = 'partially_refunded'
WHERE id = 'aa0e8400-e29b-41d4-a716-446655440006';

INSERT INTO transactions (job_id, transaction_type, from_party, to_party, amount, currency,
  stripe_payment_intent_id, status, description)
VALUES
  ('aa0e8400-e29b-41d4-a716-446655440006', 'refund', null, '550e8400-e29b-41d4-a716-446655440000',
   16.10, 'AUD', 'pi_3P2xYzAbCdEfGhIj', 'completed', 'Partial refund to family (50%)'),
  ('aa0e8400-e29b-41d4-a716-446655440006', 'provider_payout', null,
   ↪  '770e8400-e29b-41d4-a716-446655440003',
   7.00, 'AUD', 'pi_3P2xYzAbCdEfGhIj', 'completed', 'Cancellation compensation to provider (25%)'),
  ('aa0e8400-e29b-41d4-a716-446655440006', 'platform_fee', null, null,
   1.05, 'AUD', 'pi_3P2xYzAbCdEfGhIj', 'completed', 'Platform fee retained (25%)');
```

---

7. ERROR HANDLING & EDGE CASES

7.1 Payment Intent States & Actions

Stripe Status	What It Means	Your Action
`requires_payment_method`	Card not entered yet	Show payment form
`requires_confirmation`	Card entered, needs confirmation	Call `confirmCardPayment`
`requires_action`	3D Secure / bank auth required	Handle authentication
`requires_capture`	Hold placed, waiting for job completion	**NORMAL STATE —do nothing**

Table 8 – continued

Stripe Status	What It Means	Your Action
`processing`	Capture in progress	Wait for webhook
`succeeded`	Payment captured	Release to provider
`canceled`	Hold released or expired	Update job status
`payment_failed`	Card declined	Notify family, offer retry

7.2 Common Errors & Recovery

Error: `card_declined` — Generic decline

```
// Frontend: Show friendly message, suggest alternatives
{
  "error": {
    "code": "card_declined",
    "message": "Your bank declined this payment. This usually means:",
    "suggestions": [
      "Try a different card",
      "Contact your bank to authorise payments to 'Another Mother'",
      "Try again in a few minutes"
    ],
    "can_retry": true
  }
}
```

Error: `insufficient_funds`

```
{
  "error": {
    "code": "insufficient_funds",
    "message": "Your card doesn't have enough available balance for this hold.",
    "suggestions": [
      "The hold is $32.20 — ensure you have at least this amount available",
      "Try a different card with higher limit",
      "Contact your bank to check your available balance"
    ],
    "can_retry": true
  }
}
```

Error: `expired_card`

```
{
  "error": {
    "code": "expired_card",
    "message": "Your card has expired. Please update your payment method.",
    "action_required": "update_payment_method",
    "can_retry": false
  }
}
```

Error: `processing_error` — Stripe internal error

```
// Backend: Log, alert admin, auto-retry once
{
  "error": {
    "code": "processing_error",
    "message": "Something went wrong on our end. We've logged this and will retry automatically.",
    "retry_in_seconds": 30,
    "can_retry": true,
    "support_reference": "ERR-20260514-001"
  }
}
```

7.3 Idempotency for Safe Retries

Critical for payment operations:

```
// Every payment-related API call must include idempotency key
const paymentIntent = await stripe.paymentIntents.create({
  amount: 3220,
  currency: 'aud',
  // ... other params
}, {
  idempotencyKey: `job_aa0e8400-e29b-41d4-a716-446655440006_create`  // Unique per operation
});

// Same for capture, refund, transfer
const capture = await stripe.paymentIntents.capture('pi_3P2xYzAbCdEfGhIj', {
  amount_to_capture: 3220
}, {
  idempotencyKey: `job_aa0e8400-e29b-41d4-a716-446655440006_capture`
});
```

Idempotency key format:

```
{resource}_{id}_{operation}
Examples:
- job_aa0e8400-e29b-41d4-a716-446655440006_create
- job_aa0e8400-e29b-41d4-a716-446655440006_capture
- job_aa0e8400-e29b-41d4-a716-446655440006_refund
```

7.4 Dead Letter Queue for Failed Webhooks

```
// If webhook processing fails after sending 200 to Stripe
async function handleStripeEvent(event) {
  try {
    await processEvent(event);
    await markEventProcessed(event.id, 'success');
  } catch (err) {
    await markEventProcessed(event.id, 'failed');
    await enqueueDeadLetter({
      event_id: event.id,
```

```
      event_type: event.type,
      payload: event.data.object,
      error: err.message,
      retry_count: 0,
      max_retries: 5,
      next_retry_at: new Date(Date.now() + 60000)  // Retry in 1 min
    });

    // Alert if critical event
    if (isCriticalEvent(event.type)) {
      await alertAdmin('webhook_failed', { event_id: event.id, error: err.message });
    }
  }
}

// Cron job processes dead letter queue every minute
async function processDeadLetterQueue() {
  const failedEvents = await db.query(`
    SELECT * FROM webhook_dead_letter
    WHERE status = 'failed'
    AND retry_count < max_retries
    AND next_retry_at <= NOW()
    ORDER BY next_retry_at
    LIMIT 10
  `);

  for (const event of failedEvents.rows) {
    try {
      await processEvent({ type: event.event_type, data: { object: event.payload } });
      await db.query(`UPDATE webhook_dead_letter SET status = 'resolved' WHERE id = $1`, [event.id]);
    } catch (err) {
      await db.query(`
        UPDATE webhook_dead_letter
        SET retry_count = retry_count + 1,
            next_retry_at = NOW() + INTERVAL '${Math.pow(2, event.retry_count)} minutes',
            last_error = $1
        WHERE id = $2
      `, [err.message, event.id]);
    }
  }
}
```

---

8. AUSTRALIAN COMPLIANCE & TAX

8.1 GST (Goods and Services Tax)

Scenario	GST Treatment
Platform fee (15%)	**GST applicable** —Another Mother charges 10% GST on the $4.20 fee = $0.42
Provider service (85%)	**GST may apply** if provider is GST-registered
Total family charge	$32.20 + GST if applicable

Stripe Tax integration (recommended):

```
// Add Stripe Tax to automatically calculate GST
const paymentIntent = await stripe.paymentIntents.create({
  amount: 3220,
  currency: 'aud',
  automatic_tax: {
    enabled: true  // Stripe Tax calculates GST based on AU tax rules
  },
  customer_details: {
    address: {
      line1: "123 Given Terrace",
      city: "Paddington",
      state: "QLD",
      postal_code: "4064",
      country: "AU"
    }
  }
});
```

8.2 Tax Reporting

Another Mother obligations:

- Issue tax invoices to families (via email)
- Report platform fees as revenue
- GST return quarterly (if turnover > $75k)

Provider obligations (handled by Stripe):

- Stripe issues 1099-K equivalent (Payment Summary) to providers
- Providers report income on tax return
- Stripe Connect handles TFN collection for AU tax compliance

8.3 Privacy & Data Sovereignty

Requirement	Implementation
Australian data storage	VentraIP hosting (AU region)
Stripe data location	Stripe stores data in US (compliant with AU Privacy Act via standard contractual clauses)
Card data	**Never touch** —Stripe Elements handle all card data

Table 10 – continued

Requirement	Implementation
PCI compliance	Stripe handles PCI DSS Level 1 —you remain SAQ A (simplest level)

---

9. TESTING STRATEGY

9.1 Stripe Test Mode

Test API keys:

- Secret key: `sk_test_...`
- Publishable key: `pk_test_...`
- Webhook secret: `whsec_test_...`

Test cards:

Card Number	Scenario	Result
`4242 4242 4242 4242`	Success	Payment succeeds
`4000 0000 0000 0002`	Decline	`card_declined`
`4000 0000 0000 9995`	Insufficient funds	`insufficient_funds`
`4000 0000 0000 9987`	Lost card	`lost_card`
`4000 0025 0000 3155`	3D Secure required	Triggers authentication flow
`4000 0000 0000 3238`	Requires action	`requires_action`

Test Connect accounts:

```
// Create test Express account
const testAccount = await stripe.accounts.create({
  type: 'express',
  country: 'AU'
});

// Simulate verification
await stripe.accounts.update(testAccount.id, {
  tos_acceptance: { date: Math.floor(Date.now() / 1000), ip: '1.2.3.4' }
});
```

9.2 Integration Test Scenarios

Test	Expected Result
Family books job → Provider accepts → Job completes → Payment releases	Full flow < 3s from capture to transfer
Family cancels within 10 min	Full refund, no provider compensation
Family cancels 3h before job	50% refund, 25% to provider
Provider cancels after acceptance	Immediate re-assign, provider strike
Card declined during booking	Job status: `cancelled`, family notified
Webhook delivery fails	Dead letter queue retries, admin alerted
Duplicate capture attempt (idempotency)	Second attempt returns same result, no double-charge
Provider payout fails	Incident created, admin alerted, provider notified

9.3 Load Testing

Scenario	Target
100 concurrent booking attempts	All succeed, no race conditions
50 concurrent payment captures	All transfers created correctly
Webhook burst (100 events/sec)	All processed within 30s, no drops
Provider onboarding (20 concurrent)	All accounts created, no rate limit errors

---

10. MONITORING & ALERTS

10.1 Key Metrics to Track

Metric	Target	Alert If
Payment success rate	> 98%	< 95%
Time: booking → hold placed	< 5s	> 10s
Time: job complete → provider paid	< 24h	> 48h
Webhook processing latency	< 2s	> 5s
Webhook failure rate	< 0.1%	> 1%
Payout failure rate	< 0.5%	> 2%
Dispute rate	< 0.1%	> 0.5%
Connect onboarding completion	> 80%	< 70%

10.2 Alert Channels

Severity	Channel	Response Time
Critical (dispute, payout failure, fraud)	SMS + Email + PagerDuty	< 5 min
High (webhook failures, payment declines spike)	Email + Slack	< 15 min
Medium (onboarding drop-off, refund spike)	Slack	< 1 hour
Low (metrics drift, optimisation opportunities)	Weekly report	N/A

---

11. POST-MVP PAYMENT FEATURES

Feature	Description	Stripe Product
Subscription plans	Priority booking, reduced fees	Stripe Billing
Saved payment methods	One-tap rebooking	Stripe SetupIntents
Provider instant payouts	Always-on instant option	Stripe Connect Payouts
Family payment plans	Split large bookings	Stripe Installments (AU)
Gift cards	"Gift a mum" credit	Stripe Issuing
Referral credits	Account credit for referrals	Stripe Balance

---

12. COMPLETE ENVIRONMENT CONFIGURATION

12.1 Required Environment Variables

```
# Stripe
STRIPE_SECRET_KEY=sk_live_xxxxxxxxxxxxxxx
STRIPE_PUBLISHABLE_KEY=pk_live_xxxxxxxxxxxxxxx
STRIPE_WEBHOOK_SECRET=whsec_xxxxxxxxxxxxxxx
STRIPE_CONNECT_CLIENT_ID=ca_xxxxxxxxxxxxxxx  # For OAuth (if needed)

# Stripe Tax (optional, for GST)
STRIPE_TAX_ENABLED=true

# Database
DATABASE_URL=postgresql://user:pass@localhost:5432/anothermother
REDIS_URL=redis://localhost:6379

# SendGrid (for payout notifications)
SENDGRID_API_KEY=SG.xxxxxxxx

# Twilio (for SMS fallback)
TWILIO_ACCOUNT_SID=ACxxxxxxxx
TWILIO_AUTH_TOKEN=xxxxxxxx
TWILIO_PHONE_NUMBER=+61480000000
```

```
# Application
NODE_ENV=production
API_BASE_URL=https://api.anothermother.com.au/v1
FRONTEND_URL=https://anothermother.com.au
PLATFORM_FEE_PERCENTAGE=15
DEFAULT_PAYOUT_SCHEDULE=weekly
DEFAULT_PAYOUT_ANCHOR=thursday
INSTANT_PAYOUT_FEE_PERCENTAGE=1.5
```

---

13. QUICK REFERENCE: API CALL SEQUENCE

13.1 Happy Path: Full Job Lifecycle

```
[Family] POST /jobs → Backend creates PaymentIntent
[Family] stripe.confirmCardPayment(clientSecret) → Hold placed
[Stripe] webhook: payment_intent.amount_capturable_updated
[Backend] Update job → confirmed, notify provider
[Provider] POST /offers/{id}/accept → Job confirmed
[Provider] POST /jobs/{id}/status → en_route → in_progress
[Provider] POST /jobs/{id}/complete → Mark complete
[Family] POST /jobs/{id}/verify-complete → Confirm
[Backend] POST /v1/payment_intents/{id}/capture
[Backend] POST /v1/transfers → 85% to provider
[Stripe] webhook: payment_intent.succeeded
[Stripe] webhook: transfer.paid
[Stripe] webhook: payout.paid (weekly Thursday)
[Backend] Update provider earnings, send notifications
```

13.2 Cancellation Path

```
[Family] POST /jobs/{id}/cancel
[Backend] Calculate refund based on timing
[Backend] POST /v1/refunds (partial or full)
[Stripe] webhook: charge.refunded
[Backend] Update job status, log transactions
[Backend] If provider compensation needed: POST /v1/transfers
[Backend] Notify both parties
```